21, so short-circuiting doesn't always do an about-face back down the pipeline. For example, short-circuited action filters bypass only action method execution; the result filters and result execution stages still run.

The other difficultly is what happens if you have more than one filter in a stage. Let's say you have three resource filters executing in a pipeline. What happens if the second filter causes a short circuit? Any remaining filters are bypassed, but the first resource filter has already run its `*Executing` command, as shown in figure 22.2. This earlier filter gets to run its `*Executed` command too, with `context.Cancelled = true`, indicating that a filter in that stage (the resource filter stage) short-circuited the pipeline.

## Running result filters after short-circuits with `IAlwaysRunResultFilter`

Result filters are designed to wrap the execution of an `IActionResult` returned by an action method or action filter so that you can customize how the action result is executed. However, this customization doesn't apply to the `IActionResult` set when you short-circuit the filter pipeline by setting `context.Result` in an authorization filter, resource filter, or exception filter.

That's often not a problem, as many result filters are designed to handle "happy path" transformations. But sometimes you want to make sure that a transformation is always applied to an `IActionResult`, regardless of whether it was returned by an action method or a short-circuiting filter.

For those cases, you can implement `IAlwaysRunResultFilter` or `IAsyncAlwaysRunResultFilter`. These interfaces extend (and are identical) to the standard result filter interfaces, so they run like normal result filters in the filter pipeline. But these interfaces mark the filter to also run after an authorization filter, resource filter, or exception filter short-circuits the pipeline, where standard result filters *won't* run.